**Prototype with a Hand Built AI Assistant then Scaled as a Distributed System**

systems technologies, reliability measurements, and an AI assistant that helps users find opportunities and events.

## Common requirements

Both labs are pair work. Use the pair number assigned on Canvas to derive the configuration below and report all derived values in the README and report.

| Parameter | Required derivation |
|---|---|
| PAIR | Your assigned two-digit lab pair number |
| PORT_BASE | 9000 + (PAIR × 10) |
| Database and Kafka prefix | p{PAIR} |
| SEED | PAIR for every generator and random_state |
| CITY_SET | The three cities assigned to your pair on Canvas |

- One private repository per pair, tagged lab1 and lab2, with regular commits by both partners.
- Include RUN_LOG.txt and METRICS.md for each lab.
- Include AI_USE.md. Each partner must identify assistant use, show one incorrect output, explain how it was detected, and document the fix.
- Record one five-minute screen capture per lab. Both partners must run the system and explain code selected by the grader.
- Do not commit virtual environments, node_modules, __pycache__, secrets, or API keys.

## Lab 2 Scaling the Handshake platform and agent

Due date: 11/16/2026 Total: 40 points. Build on Lab 1.

Goal: Containerize the Handshake services, orchestrate them with Kubernetes, add Kafka for asynchronous application and event flows, migrate to MongoDB, integrate Redux, and measure performance with Apache JMeter. Extend the assistant with persistent memory, context management, and tracing.

Lab 2 builds directly on the Handshake platform implemented in Lab 1. Do not rebuild the core platform from scratch. Extend the Lab 1 Student, Company, Job, Application, Event, and AI Assistant services and demonstrate how they operate as a distributed system.

## Part 1 Docker and Kubernetes 11 points

### Part 1.1 Dockerize the Lab 1 Handshake services

Start from the working Lab 1 implementation. Create a separate Dockerfile for the Student, Company, Job, Application, Event, and AI Assistant services. Each Dockerfile must install only the required dependencies, start the correct service, expose its required port, and support a health check. Use the pair prefix p{PAIR} consistently when naming images, containers, and related resources.

### Part 1.2 Run and verify the complete stack locally

Use Docker Compose to run the complete Handshake stack locally. Define the services, networks, ports, environment variables, database connection, and service dependencies in the Compose file. Demonstrate that the frontend can reach the backend, the backend can reach its database, and the AI Assistant can reach the required platform services. Include the commands used to build, start, inspect, and stop the stack in the README.

### Part 1.3 Prepare the Kubernetes deployment

Create Kubernetes manifests for every service, including Deployments, Services, ConfigMaps, and Secrets where appropriate. Create a namespace beginning with p{PAIR}. Prefix all Deployment, Service, namespace, and related resource names with p{PAIR}. Configure service-to-service communication using Kubernetes service discovery. Set replica counts and resource requests or limits so that the services can scale in the cluster. Do not place passwords, access keys, or API tokens directly in committed manifests.

### Part 1.4 Deploy the Handshake platform to AWS

Use Amazon Elastic Container Registry (ECR) to store the container images and Amazon Elastic Kubernetes Service (EKS) to run the Kubernetes cluster, unless the instructor approves another AWS equivalent. Build each image, tag it with the correct ECR repository and version, authenticate securely, and push it to ECR. Create or use the assigned EKS cluster, apply the p{PAIR} namespace and manifests, and confirm that all pods reach a healthy running state.

### Part 1.5 Expose and test the AWS deployment

Expose the frontend or API through an AWS LoadBalancer Service or an approved Ingress. Configure the deployed services with environment variables and secrets using an appropriate mechanism; never

commit credentials to Git. Send one successful end-to-end request through the deployed Handshake application, such as signing in, searching for a job, viewing a posting, or submitting an application, and show the request and response as evidence.

## Part 1.6 Required AWS and Kubernetes evidence

Include screenshots showing the ECR repositories and pushed images, the EKS cluster, the p{PAIR} namespace, running pods, Kubernetes services, the LoadBalancer or Ingress endpoint, and the successful end-to-end request. Screenshots must clearly show the p{PAIR} naming convention. Include relevant kubectl output and explain how the services communicate and scale. Remove or hide access keys, passwords, tokens, and other secrets from every screenshot and report.

## Important note: AWS demonstration and viva

Lab 2 includes a live AWS demonstration and viva. Before the demonstration, each pair must complete and verify the full Handshake deployment in AWS, including the container images, Kubernetes services, Kafka flows, MongoDB integration, frontend, backend, and AI Assistant. After completing the required verification and collecting evidence, a team may pause, scale down, or temporarily stop AWS resources to control cost, but must not delete the code, manifests, data, configuration, or evidence needed to restore the system. At the scheduled demo time, the pair must bring the application back up, make it accessible, demonstrate the required workflows, and explain the architecture and implementation. Both partners must be prepared to answer questions and explain or modify code during the viva. A deployment that cannot be restored or demonstrated may not receive credit for the AWS and demonstration requirements.

## Part 2 Kafka asynchronous messaging 8 points

Deploy Kafka inside the Kubernetes cluster. Student-facing APIs publish events and worker services consume them. Include an architecture diagram in the report.

| Topic | Producer | Consumer |
|---|---|---|
| application.created / application.updated | Application API | Application Worker |
| application.status_changed | Company or Application API | Notification or Application Worker |
| event.registration_created | Event API | Event Worker |
| job.created | Job API | Search Index or Job Worker |

Demonstrate three Kafka failure cases and document each in RUN_LOG.txt: (1) publish the same event twice with the same key, make the consumer idempotent, and prove that the database does not contain a duplicate; (2) kill a worker during processing, restart it, explain whether the event was lost or reprocessed, and commit the offset only after the database write; and (3) publish a poison message, add bounded retries and a dead-letter topic, and show that the pipeline continues after the failed message is isolated.

## Part 3 MongoDB and agent memory 9 points

## MongoDB migration 4 points

Migrate student accounts, profiles, preferences, sessions, companies, job postings, applications, resumes or resume metadata, events, registrations, saved jobs and events, and activity logs to MongoDB. Keep bcrypt password protection and secure sessions with expiry.

## Three layer assistant memory 5 points

| Layer | Collection | Contents |
|---|---|---|
| Instructions | assistant_rules | Rules the assistant always follows, such as never revealing private student information and always stating eligibility requirements. |
| Learned memory | student_memory | What the assistant learns about a student across sessions, one entry per line with ISO date, category, and description. |
| Session | conversations | Current conversation only; discarded or compacted at session end. |

- Inject appropriate memory into the assistant prompt and implement a /memory command.
- Run consolidation as a Kafka consumer rather than blocking a user request.
- Seed one student memory with at least 60 entries, including at least three deliberate contradictions and three duplicate entries. Trigger consolidation and show the memory before and after. Verify that the result is below 40 entries, duplicates are merged, and contradictions are resolved. Explain one consolidation decision or correction in the report, including any case the model handled incorrectly.

## Part 4 Redux integration 4 points

- Create Redux slices for authentication and session, jobs and companies, applications, events and registrations, saved items, and the assistant conversation.
- Use actions, reducers, and selectors correctly. Show Redux DevTools screenshots for at least two slices, including the conversation slice.
- The user must be able to leave the assistant and return without losing the conversation thread.

## Part 5 JMeter performance testing 5 points

Test login, job search, application submission through Kafka, event search or registration, and the assistant endpoint.

For Login, Job Search, Application Submission, and Event Search or Registration, test concurrency levels of 100, 200, 300, 400, and 500 users. Record average response time, throughput in requests per second, and error rate at every level. Test the Assistant endpoint separately at 5, 10, 20, and 50 concurrent users, and report p50, p95, p99 latency, and error rate. Plot average response time against

concurrency and explain queueing, model inference, database work, worker capacity, and the first bottleneck observed.

| Endpoint | Concurrency | Report |
|---|---|---|
| Login | 100, 200, 300, 400, 500 | Average response time, throughput, error rate |
| Job search | 100, 200, 300, 400, 500 | Average response time, throughput, error rate |
| Application submission | 100, 200, 300, 400, 500 | Average response time, throughput, error rate |
| Assistant chat | 5, 10, 20, 50 | p50, p95, p99, error rate |

- Submit the .jmx test plan, screenshots, results, and a graph of average response time against concurrency.
- Explain bottlenecks at each level. Compare CRUD endpoints with the assistant endpoint and explain queueing, model inference, database work, and worker capacity.
- Use enough seeded data for the load test to be meaningful.

## Part 6 Context management, guardrails, and observability 3 points

- Token tracking: after every model call, record prompt and completion tokens, display tokens used and context percentage in the assistant UI, and warn above 70 percent.
- Compaction: above a chosen threshold, summarize student preferences, jobs, companies, events, applications, and current request; replace stored history with the summary; expose a command; report token counts before and after; prove recall after compaction.
- Tracing: assign every assistant request a trace_id and propagate it through Kafka events and service logs. Show one successful and one failed trace with per-stage timing. Using 50 requests, identify whether model inference, database queries, tool execution, or queue wait dominates latency.

## Agentic guardrails and evaluation harness

- Guardrail 1
  Tool allowlist: permit calls only to declared tools and reject every unknown tool name.
- Guardrail 2
  Schema validation: validate every tool call against its typed JSON schema before execution; reject malformed or extra arguments.
- Guardrail 3
  Authorization: enforce authentication, role permissions, and ownership checks for every protected operation.
- Guardrail 4
  Read/write control: allow read-only tools to run automatically, but require explicit confirmation before state-changing actions.

- Guardrail 5
  Prompt-injection defense: label and delimit retrieved job, event, profile, and database text as untrusted data; never treat it as instructions.
- Guardrail 6
  Privacy and secrets: do not reveal another student's private data; redact passwords, tokens, API keys, and other secrets from responses and logs.
- Guardrail 7
  Bounded execution: enforce request timeouts, a hard maximum iteration count, and at most one retry for a recoverable tool error.
- Guardrail 8
  Reliable answers: return clear no-match and error messages without inventing jobs, events, companies, or users.
- ***Harness requirement:*** build a repeatable test harness using a mock or deterministic model client where appropriate; it must exercise the same production guardrail, validation, authorization, and tool-execution modules as the deployed assistant.
- The harness must test: a successful read-only search; an accepted state-changing action; a declined state-changing action; malformed arguments; an unknown tool; a no-match request; a conversation-history follow-up; prompt-injection content before and after the defense; the iteration ceiling or timeout; and one recovery retry.
- For every harness case, record the input, assistant messages, tool names and arguments, tool results, final answer, guardrail outcome, errors or retries, iteration and tool-call counts, and elapsed time.
- Produce machine-readable output such as agent_harness.json or agent_harness.jsonl and a human-readable summary in METRICS.md or the report.
- Report pass/fail counts, reliability before and after repair, one blocked injection, one declined confirmation, one repaired malformed call, and one bounded-stop case.
- Make the harness runnable from the README with one command. Keep the required real Ollama verification transcripts; the harness supplements those transcripts and does not replace them.

## Lab 2 submission

- Update the private repository with Dockerfiles, Docker Compose configuration, Kubernetes manifests, AWS ECR and EKS deployment configuration, Kafka producers and consumers, MongoDB migration, Redux implementation, JMeter test plans and results, agent guardrails, evaluation harness source and output, and guardrail evidence. Include screenshots of the services running on AWS, ECR images, the EKS cluster, the deployed endpoint, Kafka message flows, Redux DevTools, and the agentic safety tests. Do not commit virtual environments, __pycache__, credentials, or API keys. Include requirements.txt and update the README with instructions for running locally, running the harness, and accessing the AWS deployment.
- Submit LabPair-##_Lab2_Report.pdf covering architecture, cloud deployment, Kafka flows and failures, MongoDB schema, memory and consolidation, Redux state, performance graphs, assistant load analysis, token compaction evidence, and one end-to-end trace. Include the agent guardrail results, harness test matrix and output, and blocked-injection, declined-confirmation, repaired-call, and bounded-stop evidence.

- Include METRICS.md, RUN_LOG.txt, AI_USE.md, and the pair verification recording

**Grading rubric**

| Part | Criteria | Points |
|---|---|---|
| 1 | Dockerfiles for all services including the assistant (4); Docker Compose runs the stack locally (2); Kubernetes manifests (3); services communicate in cluster (1); AWS ECR/EKS deployment and evidence (1) | 11 |
| 2 | Kafka deployment 1; asynchronous flows 2; architecture 2; failure scenarios and fixes 3 | 8 |
| 3 | MongoDB migration and security 4; memory injection and command 2; consolidation consumer and verification 3 | 9 |
| 4 | Required Redux slices 2; actions and selectors 1; DevTools evidence 1 | 4 |
| 5 | JMeter plan 1; CRUD load tests 2; assistant load tests and analysis 2 | 5 |
| 6 | Token tracking 1; compaction and recall 1; trace propagation, guardrails, harness, and stage timing 1 | 3 |
| Total | | 40 |

## Report checklist

| Evidence | Lab 1 | Lab 2 |
|---|---|---|
| README and run instructions | Required | Updated |
| Screenshots and API tests | Required | Cloud and Kafka evidence |
| METRICS.md and RUN_LOG.txt | Required | Updated with failures and traces |
| AI_USE.md and recording | Required | Required |
| Pair parameters and seed counts | Required | Required |
| Git history with both partners | Required | Updated |